Clase 4: Pipes y Redirección - Conectando Procesos

Introducción: La filosofía UNIX

Una de las ideas más poderosas de UNIX es que los programas deberían hacer una cosa bien y comunicarse entre sí. En lugar de crear programas monolíticos que hacen todo, UNIX favorece herramientas pequeñas y especializadas que pueden combinarse como bloques de LEGO.

Pero para que esta filosofía funcione, los procesos necesitan una forma de comunicarse. ¿Cómo pasa la salida de un programa a la entrada de otro? ¿Cómo guardamos la salida en un archivo? ¿Cómo procesamos datos que vienen de diferentes fuentes?

La respuesta está en los **file descriptors**, la **redirección**, y los **pipes**. Estos mecanismos son fundamentales para entender cómo funciona un shell y cómo los procesos cooperan en UNIX.

File Descriptors: la abstracción fundamental

Todo es un archivo

En UNIX, casi todo se representa como un archivo: archivos regulares, directorios, dispositivos, sockets de red, pipes. Esta abstracción unificada significa que las mismas operaciones (`read`, `write`, `close`) funcionan con todos ellos.

Cuando un proceso quiere trabajar con un "archivo" (en el sentido amplio), el kernel le da un número entero llamado **file descriptor (fd)**. Este número es simplemente un índice en una tabla que el kernel mantiene para cada proceso.

Los tres file descriptors estándar

Todo proceso en UNIX nace con tres file descriptors ya abiertos:

fd	Nombre	Propósito	Constante
0	stdin	Entrada estándar	`sys.stdin`
1	stdout	Salida estándar	`sys.stdout`
2	stderr	Salida de errores	`sys.stderr`

Por defecto, estos tres están conectados a la terminal. Cuando escribís en stdout, aparece en la pantalla. Cuando leés de stdin, leés del teclado.

```
import sys
import os

# Ver los file descriptors
print(f"stdin  fileno: {sys.stdin.fileno()}")   # 0
print(f"stdout fileno: {sys.stdout.fileno()}")  # 1
print(f"stderr fileno: {sys.stderr.fileno()}")  # 2

# También podemos verlos directamente
print(f"stdin  es fd {os.dup(0)}")  # dup retorna un nuev
```

La tabla de file descriptors

Cada proceso tiene su propia tabla de file descriptors. Esta tabla es un array donde cada índice (el fd) apunta a una estructura del kernel que describe el archivo abierto.

```
Proceso A:
fd 0 -> terminal (lectura)
fd 1 -> terminal (escritura)
fd 2 -> terminal (escritura)
fd 3 -> /home/user/data.txt
fd 4 -> socket TCP a servidor

Proceso B (diferente tabla):
fd 0 -> terminal
fd 1 -> archivo.log
fd 2 -> terminal
fd 3 -> pipe (lectura)
```

Cuando hacés `open()`, el kernel busca el primer fd libre (generalmente el más bajo) y lo asigna al nuevo archivo.

```
import os

# Abrir un archivo
fd = os.open("/tmp/test.txt", os.O_CREAT | os.O_WRONLY, 0
print(f"Nuevo archivo usa fd {fd}")  # Probablemente 3

# Escribir usando el fd
os.write(fd, b"Hola desde fd\n")

# Cerrar
os.close(fd)
```

Redirección: cambiando el destino

El concepto

La redirección permite cambiar a dónde apuntan stdin, stdout o stderr sin modificar el programa. El programa sigue escribiendo a "stdout" (fd

1), pero el shell configuró ese fd para que apunte a un archivo en lugar de la terminal.

```
# stdout va a archivo
ls -la > listado.txt

# stdout se agrega al archivo
echo "otra linea" >> listado.txt

# stdin viene de archivo
wc -l < listado.txt

# stderr va a archivo
find / -name "*.conf" 2> errores.txt

# stdout y stderr al mismo archivo
comando > todo.txt 2>&1
```

dup2: la syscall detrás de la redirección

La magia de la redirección está en la syscall `dup2(oldfd, newfd)`. Esta syscall hace que `newfd` apunte al mismo lugar que `oldfd`.

```
import os
import sys

# Ejemplo: redirigir stdout a un archivo

# 1. Abrir el archivo destino
archivo = os.open("/tmp/salida.txt", os.O_CREAT | os.O_WR

# 2. Guardar una copia de stdout original (por si queremo
stdout_backup = os.dup(1)

# 3. Hacer que fd 1 (stdout) apunte al archivo
os.dup2(archivo, 1)

# 4. Cerrar el fd original del archivo (ya no lo necesita
```

```
os.close(archivo)

# 5. Ahora todo lo que va a stdout va al archivo
print("Esta línea va al archivo, no a la terminal")
os.write(1, b"Esto también va al archivo\n")

# 6. Restaurar stdout original
os.dup2(stdout_backup, 1)
os.close(stdout_backup)

print("Esta línea vuelve a la terminal")
```

Implementando redirección en un shell

Cuando escribís `ls > archivo.txt`, el shell hace esto:

1. Fork para crear un proceso hijo
2. En el hijo, antes del exec:
    - Abrir "archivo.txt" para escritura
    - Usar dup2 para que fd 1 apunte al archivo
    - Cerrar el fd original del archivo
3. Exec "ls"
4. El padre espera con wait

```
import os

def ejecutar_con_redireccion_salida(comando, args, archiv
    """Ejecuta un comando redirigiendo stdout a un archiv
    pid = os.fork()

    if pid == 0:
        # Hijo: configurar redirección antes del exec

        # Abrir archivo para stdout
        fd = os.open(archivo_salida, os.O_CREAT | os.O_WR

        # Redirigir stdout (fd 1) al archivo
        os.dup2(fd, 1)
```

```
        os.close(fd)

        # Ejecutar comando
        os.execvp(comando, [comando] + args)
        os._exit(1)
    else:
        os.wait()

# Equivalente a: ls -la /tmp > /tmp/listado.txt
ejecutar_con_redireccion_salida("ls", ["-la", "/tmp"], "/
```

Pipes: comunicación entre procesos

El concepto del pipe

Un pipe es un canal de comunicación unidireccional entre procesos. Los datos que un proceso escribe en un extremo del pipe pueden ser leídos por otro proceso en el otro extremo.

Pensalo como una tubería (literalmente "pipe" en inglés): el agua entra por un lado y sale por el otro. Los datos funcionan igual: bytes entran por el extremo de escritura y salen por el extremo de lectura, en el mismo orden (FIFO).

Creando un pipe

La syscall `pipe()` crea un pipe y retorna dos file descriptors:

```
import os

# Crear pipe
read_fd, write_fd = os.pipe()

print(f"Extremo de lectura: fd {read_fd}")
```

```
print(f"Extremo de escritura: fd {write_fd}")

# Escribir en un extremo
os.write(write_fd, b"Hola por el pipe!\n")

# Leer del otro extremo
datos = os.read(read_fd, 1024)
print(f"Leído: {datos.decode()}")

# Cerrar
os.close(read_fd)
os.close(write_fd)
```

Pipe entre padre e hijo

Un pipe se vuelve útil cuando lo combinamos con fork. El padre y el hijo pueden comunicarse a través del pipe:

```
import os

# Crear pipe ANTES del fork
read_fd, write_fd = os.pipe()

pid = os.fork()

if pid == 0:
    # Hijo: va a escribir, no necesita leer
    os.close(read_fd)

    mensaje = f"Hola padre! Soy el hijo (PID {os.getpid()
    os.write(write_fd, mensaje.encode())

    os.close(write_fd)
    os._exit(0)
else:
    # Padre: va a leer, no necesita escribir
    os.close(write_fd)
```

```
    # Leer lo que el hijo envió
    datos = os.read(read_fd, 1024)
    print(f"Padre recibió: {datos.decode()}")

    os.close(read_fd)
    os.wait()
```

Punto crucial: Cada proceso debe cerrar el extremo del pipe que no usa. Esto es importante porque:

1. Evita desperdiciar file descriptors
2. Permite que el lector detecte EOF cuando todos los escritores cierran su extremo
3. Permite que el escritor reciba SIGPIPE si no hay lectores

Conectando dos comandos: el pipe del shell

Cuando escribís `ls | grep txt` en el shell, esto es lo que sucede:

```
import os

def ejecutar_pipeline(cmd1, args1, cmd2, args2):
    """Ejecuta cmd1 | cmd2"""

    # Crear el pipe
    read_fd, write_fd = os.pipe()

    # Primer hijo: ejecuta cmd1 con stdout -> pipe
    pid1 = os.fork()
    if pid1 == 0:
        os.close(read_fd)      # No lee del pipe
        os.dup2(write_fd, 1)   # stdout -> pipe write
        os.close(write_fd)     # Ya duplicado, cerrar orig
        os.execvp(cmd1, [cmd1] + args1)
        os._exit(1)

    # Segundo hijo: ejecuta cmd2 con stdin <- pipe
    pid2 = os.fork()
```

```
    if pid2 == 0:
        os.close(write_fd)     # No escribe al pipe
        os.dup2(read_fd, 0)    # stdin <- pipe read
        os.close(read_fd)      # Ya duplicado, cerrar orig
        os.execvp(cmd2, [cmd2] + args2)
        os._exit(1)

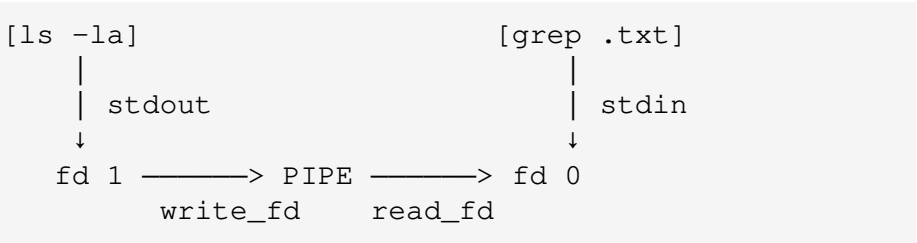
    # Padre: cerrar ambos extremos del pipe y esperar
    os.close(read_fd)
    os.close(write_fd)

    os.waitpid(pid1, 0)
    os.waitpid(pid2, 0)

# Equivalente a: ls -la | grep ".txt"
ejecutar_pipeline("ls", ["-la"], "grep", [".txt"])
```

El flujo de datos

Visualicemos el flujo:



El shell configura todo antes del exec. Una vez que `ls` hace exec, no sabe ni le importa que su stdout va a un pipe en lugar de la terminal. Simplemente escribe a fd 1 como siempre.

Buffers y comportamiento del pipe

El buffer del pipe

Los pipes tienen un buffer interno en el kernel (típicamente 64KB en Linux). Esto significa que el escritor puede escribir datos sin que el lector los lea inmediatamente.

```
import os

read_fd, write_fd = os.pipe()

# Cuánto podemos escribir antes de bloquearnos?
import fcntl
import array

# En Linux, podemos consultar el tamaño del buffer
buf = array.array('i', [0])
fcntl.ioctl(write_fd, 0x80045270, buf)  # FIONREAD consta
print(f"Tamaño típico del buffer del pipe: ~64KB")
```

Comportamiento de bloqueo

Los pipes tienen comportamiento de bloqueo importante:

1. **Lectura de pipe vacío:** Si el pipe está vacío y hay escritores, `read()` bloquea hasta que lleguen datos.
2. **Lectura de pipe sin escritores:** Si el pipe está vacío y NO hay escritores (todos cerraron su extremo), `read()` retorna 0 (EOF).
3. **Escritura a pipe lleno:** Si el buffer está lleno, `write()` bloquea hasta que el lector haga espacio.
4. **Escritura a pipe sin lectores:** Si no hay lectores, el proceso recibe SIGPIPE (por defecto lo mata).

```
import os
import signal
```

```
# Manejar SIGPIPE para ver qué pasa
def handler(sig, frame):
    print(f"Recibí SIGPIPE! No hay nadie leyendo el pipe.
    os._exit(1)

signal.signal(signal.SIGPIPE, handler)

# Crear pipe y cerrar el lado de lectura
r, w = os.pipe()
os.close(r)  # Cerrar lectura

# Intentar escribir - recibiremos SIGPIPE
try:
    os.write(w, b"Hola?")
except BrokenPipeError:
    print("BrokenPipeError: el pipe está roto")
```

Redirección avanzada

Combinando stdout y stderr

A veces queremos capturar tanto la salida normal como los errores:

```
# Ambos al mismo archivo
comando > todo.txt 2>&1

# A diferentes archivos
comando > salida.txt 2> errores.txt

# stderr a stdout (para procesar con pipe)
comando 2>&1 | grep "error"
```

En Python:

```
import os

def ejecutar_capturando_todo(comando, args, archivo):
    """Ejecuta comando redirigiendo stdout y stderr al mi
    pid = os.fork()

    if pid == 0:
        # Abrir archivo
        fd = os.open(archivo, os.O_CREAT | os.O_WRONLY |

        # Redirigir stdout
        os.dup2(fd, 1)

        # Redirigir stderr al mismo lugar que stdout
        os.dup2(1, 2)  # stderr ahora apunta a donde apun

        os.close(fd)

        os.execvp(comando, [comando] + args)
        os._exit(1)
    else:
        os.wait()
```

Here documents y here strings

El shell permite alimentar stdin desde texto inline:

```
# Here document (múltiples líneas)
cat << EOF
Línea 1
Línea 2
Línea 3
EOF

# Here string (una línea)
grep "patron" <<< "texto a buscar"
```

En Python podemos simular esto con pipes:

```
import os

def ejecutar_con_input(comando, args, texto_input):
    """Ejecuta comando pasándole texto como stdin."""
    read_fd, write_fd = os.pipe()

    pid = os.fork()

    if pid == 0:
        os.close(write_fd)    # Hijo no escribe
        os.dup2(read_fd, 0)   # stdin <- pipe
        os.close(read_fd)
        os.execvp(comando, [comando] + args)
        os._exit(1)
    else:
        os.close(read_fd)     # Padre no lee
        os.write(write_fd, texto_input.encode())
        os.close(write_fd)    # Importante: cerrar para q
        os.wait()

# Equivalente a: grep "error" <<< "linea 1\nerror aqui\nl
ejecutar_con_input("grep", ["error"], "linea 1\nerror aqu
```

Subprocess y pipes

El módulo `subprocess` facilita enormemente el trabajo con pipes:

```
import subprocess

# Capturar salida
resultado = subprocess.run(
    ["ls", "-la"],
    capture_output=True,
    text=True
)
```

```
print(resultado.stdout)

# Pasar input
resultado = subprocess.run(
    ["grep", "error"],
    input="linea 1\nerror aqui\nlinea 3\n",
    capture_output=True,
    text=True
)
print(resultado.stdout)

# Pipeline con subprocess
# ls -la | grep ".txt" | wc -l

ls = subprocess.Popen(
    ["ls", "-la"],
    stdout=subprocess.PIPE
)

grep = subprocess.Popen(
    ["grep", ".txt"],
    stdin=ls.stdout,
    stdout=subprocess.PIPE
)

wc = subprocess.Popen(
    ["wc", "-l"],
    stdin=grep.stdout,
    stdout=subprocess.PIPE,
    text=True
)

# Cerrar los pipes del padre para que los procesos vean E
ls.stdout.close()
grep.stdout.close()

salida, _ = wc.communicate()
print(f"Archivos .txt encontrados: {salida.strip()}")
```

La opción shell=True

`subprocess` puede usar el shell para interpretar el comando:

```
import subprocess

# Con shell=True el shell interpreta pipes, redirección,
resultado = subprocess.run(
    "ls -la | grep .txt | wc -l",
    shell=True,
    capture_output=True,
    text=True
)
print(resultado.stdout)
```

Advertencia de seguridad: Nunca uses `shell=True` con input del usuario. Es vulnerable a inyección de comandos:

```
# PELIGROSO - nunca hacer esto con input de usuario
usuario = input("Buscar archivo: ")
subprocess.run(f"find / -name {usuario}", shell=True)  #

# El usuario podría ingresar: "x; rm -rf /"
```

Named pipes (FIFOs)

Los pipes normales (anonymous pipes) solo pueden conectar procesos relacionados (padre-hijo o hermanos). Los **named pipes** o **FIFOs** son pipes con nombre en el filesystem, permitiendo que procesos no relacionados se comuniquen.

```
import os
import stat

# Crear named pipe
```

```
fifo_path = "/tmp/mi_fifo"

try:
    os.mkfifo(fifo_path)
except FileExistsError:
    pass

print(f"FIFO creado en {fifo_path}")
print("Ejecutá 'cat /tmp/mi_fifo' en otra terminal para l

# Escribir al FIFO (bloquea hasta que haya un lector)
with open(fifo_path, 'w') as f:
    f.write("Hola desde el escritor!\n")
    f.write("Esto va por el named pipe\n")

print("Datos enviados!")
```

En otra terminal:

```
cat /tmp/mi_fifo
# Verás: Hola desde el escritor!
#        Esto va por el named pipe
```

Diferencias con pipes anónimos

Característica	Pipe anónimo	Named pipe (FIFO)
Nombre en filesystem	No	Sí
Procesos conectables	Relacionados	Cualquiera
Creación	`pipe()`	`mkfifo()`
Persistencia	Muere con procesos	Persiste hasta borrarlo
Bidireccional	No (un sentido)	No (un sentido)

Conceptos clave

1. **File descriptors** son índices en la tabla de archivos abiertos de cada proceso. stdin=0, stdout=1, stderr=2.
2. **dup2(old, new)** hace que `new` apunte al mismo lugar que `old`. Es la base de la redirección.
3. **pipe()** crea un canal unidireccional de comunicación, retornando dos fds (lectura, escritura).
4. **El patrón de pipeline:** fork, configurar pipes/redirección con dup2, exec. El padre cierra sus copias de los fds del pipe.
5. **Bloqueo:** read de pipe vacío bloquea; read de pipe sin escritores retorna 0; write a pipe sin lectores genera SIGPIPE.
6. **subprocess** abstrae todo esto con una API de alto nivel. Preferilo para código de producción.

Preparación para la próxima clase

En la clase 5 profundizaremos en **señales** - el mecanismo de notificaciones asíncronas entre procesos. Las señales permiten manejar eventos como Ctrl+C, terminar procesos limpiamente, y coordinar acciones entre procesos de formas que los pipes no pueden.
